Bubble sort

```
import time

# Function for Bubble Sort
def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        swapped = False

        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap adjacent elements
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # Stop if the array is already sorted
        if not swapped:
            break


# User Input
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Start timer
start_time = time.perf_counter()

# Perform Bubble Sort
bubble_sort(arr)

# End timer
end_time = time.perf_counter()

# Calculate execution time
execution_time = end_time - start_time

# Display results
print("\nSorted Array:")
print(arr)

print(f"\nExecution Time: {execution_time:.8f} seconds")

print("\nTime Complexity:")
print("Best Case    : O(n)")
print("Average Case : O(n²)")
print("Worst Case   : O(n²)")

print("\nSpace Complexity: O(1)")
```

```
Enter the number of elements: 5
Enter the elements:
87
7
36
49
91

Sorted Array:
[7, 36, 49, 87, 91]

Execution Time: 0.00010953 seconds

Time Complexity:
Best Case    : O(n)
Average Case : O(n²)
Worst Case   : O(n²)

Space Complexity: O(1)
```

Selection sort

```
import time

# Function for Selection Sort
```

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        # Assume the current element is the minimum
        min_index = i

        # Find the index of the minimum element
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        # Swap the found minimum element with the first unsorted element
        arr[i], arr[min_index] = arr[min_index], arr[i]


# User Input
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Start timer
start_time = time.perf_counter()

# Perform Selection Sort
selection_sort(arr)

# End timer
end_time = time.perf_counter()

# Calculate execution time
execution_time = end_time - start_time

# Display results
print("\nSorted Array:")
print(arr)

print(f"\nExecution Time: {execution_time:.8f} seconds")

print("\nTime Complexity:")
print("Best Case    : O(n²)")
print("Average Case : O(n²)")
print("Worst Case   : O(n²)")

print("\nSpace Complexity: O(1)")
```

```
Enter the number of elements: 5
Enter the elements:
12
36
77
54
21

Sorted Array:
[12, 21, 36, 54, 77]

Execution Time: 0.00008699 seconds

Time Complexity:
Best Case    : O(n²)
Average Case : O(n²)
Worst Case   : O(n²)

Space Complexity: O(1)
```

Insertion sort

```
import time

# Function for Insertion Sort
def insertion_sort(arr):
    n = len(arr)

    for i in range(1, n):
        key = arr[i]
        j = i - 1

        # Move elements that are greater than key
        # to one position ahead of their current position
```

```
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key


# User Input
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Start timer
start_time = time.perf_counter()

# Perform Insertion Sort
insertion_sort(arr)

# End timer
end_time = time.perf_counter()

# Calculate execution time
execution_time = end_time - start_time

# Display results
print("\nSorted Array:")
print(arr)

print(f"\nExecution Time: {execution_time:.8f} seconds")

print("\nTime Complexity:")
print("Best Case    : O(n)")
print("Average Case : O(n²)")
print("Worst Case   : O(n²)")

print("\nSpace Complexity: O(1)")
```

```
Enter the number of elements: 5
Enter the elements:
33
44
55
66
77

Sorted Array:
[33, 44, 55, 66, 77]

Execution Time: 0.00016098 seconds

Time Complexity:
Best Case    : O(n)
Average Case : O(n²)
Worst Case   : O(n²)

Space Complexity: O(1)
```

Merge sort

```
import time

# Function to merge two halves
def merge(arr, left, mid, right):
    left_part = arr[left:mid + 1]
    right_part = arr[mid + 1:right + 1]

    i = 0
    j = 0
    k = left

    while i < len(left_part) and j < len(right_part):
        if left_part[i] <= right_part[j]:
            arr[k] = left_part[i]
            i += 1
        else:
            arr[k] = right_part[j]
            j += 1
        k += 1

    while i < len(left_part):
```

```
            arr[k] = left_part[i]
            i += 1
            k += 1

        while j < len(right_part):
            arr[k] = right_part[j]
            j += 1
            k += 1


    # Function for Merge Sort
    def merge_sort(arr, left, right):
        if left < right:
            mid = (left + right) // 2

            merge_sort(arr, left, mid)
            merge_sort(arr, mid + 1, right)

            merge(arr, left, mid, right)


    # User Input
    n = int(input("Enter the number of elements: "))

    arr = []
    print("Enter the elements:")
    for i in range(n):
        arr.append(int(input()))

    # Start timer
    start_time = time.perf_counter()

    # Perform Merge Sort
    merge_sort(arr, 0, len(arr) - 1)

    # End timer
    end_time = time.perf_counter()

    # Calculate execution time
    execution_time = end_time - start_time

    # Display results
    print("\nSorted Array:")
    print(arr)

    print(f"\nExecution Time: {execution_time:.8f} seconds")

    print("\nTime Complexity:")
    print("Best Case    : O(n log n)")
    print("Average Case : O(n log n)")
    print("Worst Case   : O(n log n)")

    print("\nSpace Complexity: O(n)")
```

```
Enter the number of elements: 5
Enter the elements:
23
45
67
3
15

Sorted Array:
[3, 15, 23, 45, 67]

Execution Time: 0.00012513 seconds

Time Complexity:
Best Case    : O(n log n)
Average Case : O(n log n)
Worst Case   : O(n log n)

Space Complexity: O(n)
```

Quick sort

```
import time

# Function to partition the array
def partition(arr, low, high):
    pivot = arr[high]  # Choose the last element as pivot
    i = low - 1
```

```
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1


# Function for Quick Sort
def quick_sort(arr, low, high):
    if low < high:
        # Find partition index
        pi = partition(arr, low, high)

        # Recursively sort elements before and after partition
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)


# User Input
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Start timer
start_time = time.perf_counter()

# Perform Quick Sort
quick_sort(arr, 0, len(arr) - 1)

# End timer
end_time = time.perf_counter()

# Calculate execution time
execution_time = end_time - start_time

# Display results
print("\nSorted Array:")
print(arr)

print(f"\nExecution Time: {execution_time:.8f} seconds")

print("\nTime Complexity:")
print("Best Case    : O(n log n)")
print("Average Case : O(n log n)")
print("Worst Case   : O(n²)")

print("\nSpace Complexity: O(log n)")
```

```
Enter the number of elements: 5
Enter the elements:
34
58
94
5
8

Sorted Array:
[5, 8, 34, 58, 94]

Execution Time: 0.00012774 seconds

Time Complexity:
Best Case    : O(n log n)
Average Case : O(n log n)
Worst Case   : O(n²)

Space Complexity: O(log n)
```